Heaps and priority queues in Go

Contents

A heap is a complete binary tree where every parent satisfies an ordering with its children: min-heap (parent ≤ children) or max-heap (parent ≥ children). It's the data structure behind priority queues and dozens of algorithms.

Go ships `container/heap` in the stdlib. It's not a heap type; it's an interface + algorithms that operate on anything you implement.

1. Why heaps

The heap gives you:

Operation	Time
Peek min/max	O(1)
Push	O(log n)
Pop min/max	O(log n)
Build heap from N elements	O(n)
Random access	not supported

Tradeoff vs sorted slice: insertions are O(log n) instead of O(n). Tradeoff vs balanced BST: simpler, faster constants, but no “find arbitrary element” support.

Use when you repeatedly extract the smallest (or largest) element while also pushing new ones.

2. Array representation

A heap is stored as a flat slice. For node at index `i`:

```
parent = (i - 1) / 2
left   = 2*i + 1
right  = 2*i + 2
```

No pointers. Excellent cache behavior. Memory is just `len * sizeof(T)` plus the slice header.

The “complete” invariant (filled left-to-right at every level) makes this layout exact: index 0 is the root; the last node is at `len-1`.

3. container/heap

The package provides four functions:

```
heap.Init(h)                 // O(n): build heap from arbitrary state
heap.Push(h, x)              // O(log n): add element
heap.Pop(h) any              // O(log n): remove min/max
heap.Fix(h, i)               // O(log n): repair after modifying element i
heap.Remove(h, i) any        // O(log n): remove arbitrary element by index
```

You implement an interface:

```
type Interface interface {
    sort.Interface             // Len, Less, Swap
    Push(x any)                // append to the slice
    Pop() any                  // remove and return the last element
}
```

The interface is unusual: `Push` and `Pop` here mean “append/remove last,” not “heap push/pop.” The `heap` package wraps them with the heap reordering logic. You never call `(*MyHeap).Push` directly; you call `heap.Push(h, x)`.

Concrete min-heap of int

```go
type IntHeap []int

func (h IntHeap) Len() int           { return len(h) }
func (h IntHeap) Less(i, j int) bool { return h[i] < h[j] }     // min-heap
func (h IntHeap) Swap(i, j int)      { h[i], h[j] = h[j], h[i] }

func (h *IntHeap) Push(x any) {
    *h = append(*h, x.(int))
}

func (h *IntHeap) Pop() any {
    old := *h
    n := len(old)
    x := old[n-1]
    *h = old[:n-1]
    return x
}

// usage
h := &IntHeap{2, 1, 5}
heap.Init(h)                  // heap-ify in place
heap.Push(h, 3)
heap.Pop(h)                   // returns 1 (smallest)
```

Max-heap by flipping Less

```go
func (h MaxHeap) Less(i, j int) bool { return h[i] > h[j] }
```

That's the only change.

4. Generic priority queue

For real code, a generic version is cleaner:

```go
type PQItem[T any] struct {
    Value    T
    Priority int
    Index    int                  // used by heap.Remove and heap.Fix
}

type PQ[T any] []*PQItem[T]

func (pq PQ[T]) Len() int           { return len(pq) }
func (pq PQ[T]) Less(i, j int) bool { return pq[i].Priority < pq[j].Priority }
func (pq PQ[T]) Swap(i, j int) {
    pq[i], pq[j] = pq[j], pq[i]
    pq[i].Index = i
    pq[j].Index = j
}

func (pq *PQ[T]) Push(x any) {
    n := len(*pq)
    item := x.(*PQItem[T])
    item.Index = n
    *pq = append(*pq, item)
}

func (pq *PQ[T]) Pop() any {
    old := *pq
    n := len(old)
    item := old[n-1]
    old[n-1] = nil
    item.Index = -1
    *pq = old[:n-1]
    return item
}

func (pq *PQ[T]) Update(item *PQItem[T], value T, priority int) {
    item.Value = value
    item.Priority = priority
    heap.Fix(pq, item.Index)
}
```

The `Index` field is the trick that lets you reorder or remove a specific item in O(log n). Without it, you'd have to scan to find the item first.

Usage:

```
pq := &PQ[string]{}
heap.Init(pq)

a := &PQItem[string]{Value: "task A", Priority: 5}
heap.Push(pq, a)
heap.Push(pq, &PQItem[string]{Value: "task B", Priority: 1})

next := heap.Pop(pq).(*PQItem[string])
// next.Value == "task B"

pq.Update(a, "task A revised", 0)
next = heap.Pop(pq).(*PQItem[string])
// next.Value == "task A revised"
```

5. Use cases

Top-K

Find the K largest (or smallest) elements in a stream of N items.

```
func TopK(stream []int, k int) []int {
    h := &IntHeap{}            // min-heap
    heap.Init(h)
    for _, v := range stream {
        if h.Len() < k {
            heap.Push(h, v)
        } else if v > (*h)[0] {
            (*h)[0] = v
            heap.Fix(h, 0)
        }
    }
    return *h
}
```

The heap holds the K largest seen so far. The root is the smallest of those, so any new value greater than the root is a candidate. O(N log K) total.

Dijkstra’s shortest path

Classic application. The priority queue holds (distance, node) pairs; the smallest distance always pops first.

```go
func Dijkstra(graph [][]Edge, start int) []int {
    dist := make([]int, len(graph))
    for i := range dist { dist[i] = math.MaxInt }
    dist[start] = 0

    pq := &PQ[int]{}
    heap.Push(pq, &PQItem[int]{Value: start, Priority: 0})

    for pq.Len() > 0 {
        u := heap.Pop(pq).(*PQItem[int])
        if u.Priority > dist[u.Value] { continue }   // stale entry
        for _, e := range graph[u.Value] {
            if alt := dist[u.Value] + e.Weight; alt < dist[e.To] {
                dist[e.To] = alt
                heap.Push(pq, &PQItem[int]{Value: e.To, Priority: alt})
            }
        }
    }
    return dist
}
```

Note the “stale entry” check. We push a new entry rather than updating an existing one; on pop, we skip entries whose recorded distance no longer matches the current best. This is simpler than maintaining indexes and almost always faster in practice.

Event-driven scheduling

A timer service that fires events at scheduled times.

```go
type Event struct {
    At time.Time
    Fn func()
}

type Scheduler struct {
    pq    []*Event
    timer *time.Timer
}

// pq Less compares At
```

On `Schedule`, push the event. On the next earliest-event time, fire and pop. This is roughly how a single-goroutine scheduler works.

k-way merge

Merging k sorted streams into one. Push the head of each stream; pop the smallest; advance that stream and push its next element.

```go
func KWayMerge(streams [][]int) []int {
    type pos struct { stream, idx int }
    pq := &PQ[pos]{}
    for i, s := range streams {
        if len(s) > 0 {
            heap.Push(pq, &PQItem[pos]{Value: pos{i, 0}, Priority: s[0]})
        }
    }
    var out []int
    for pq.Len() > 0 {
        top := heap.Pop(pq).(*PQItem[pos])
        out = append(out, top.Priority)
        p := top.Value
        if p.idx+1 < len(streams[p.stream]) {
            heap.Push(pq, &PQItem[pos]{
                Value:    pos{p.stream, p.idx + 1},
                Priority: streams[p.stream][p.idx+1],
            })
        }
    }
    return out
}
```

O(N log k) where N is total elements and k is number of streams. Used in external sorts and database merges.

Median maintenance (two heaps)

Maintain the median of a stream by keeping a max-heap of the lower half and a min-heap of the upper half.

```go
type MedianFinder struct {
    lower MaxHeap   // smaller half
    upper MinHeap   // larger half
}

func (m *MedianFinder) Add(v int) {
    if m.lower.Len() == 0 || v <= m.lower[0] {
        heap.Push(&m.lower, v)
    } else {
        heap.Push(&m.upper, v)
    }
    // rebalance
    if m.lower.Len() > m.upper.Len()+1 {
        heap.Push(&m.upper, heap.Pop(&m.lower))
    } else if m.upper.Len() > m.lower.Len() {
        heap.Push(&m.lower, heap.Pop(&m.upper))
    }
}

func (m *MedianFinder) Median() float64 {
    if m.lower.Len() > m.upper.Len() {
        return float64(m.lower[0])
    }
    return (float64(m.lower[0]) + float64(m.upper[0])) / 2
}
```

Constant-time median, O(log n) inserts. Used in real-time analytics.

Huffman coding

Build a Huffman tree by repeatedly popping the two smallest-weight subtrees and merging them. Heap gives you the smallest in O(log n).

6. Custom comparators

For a heap that orders by a custom field, the `Less` method does everything.

```go
type Task struct {
    ID       string
    Deadline time.Time
}

type TaskHeap []*Task

func (h TaskHeap) Len() int { return len(h) }
func (h TaskHeap) Less(i, j int) bool {
    return h[i].Deadline.Before(h[j].Deadline)
}
func (h TaskHeap) Swap(i, j int) { h[i], h[j] = h[j], h[i] }
// Push / Pop as before
```

For ties, add a second criterion in `Less`:

```
func (h TaskHeap) Less(i, j int) bool {
    if !h[i].Deadline.Equal(h[j].Deadline) {
        return h[i].Deadline.Before(h[j].Deadline)
    }
    return h[i].ID < h[j].ID
}
```

This stabilizes the order so test results are reproducible.

7. Increase-key and decrease-key

Standard heap algorithms textbook talks about “decrease-key” (lower a value’s priority, then float it up). The Go interface implements this through `heap.Fix`:

```
// Lower the priority of item i, then fix
items[i].Priority = newPriority
heap.Fix(h, i)
```

This requires knowing `i`. That’s what the `Index` field on `PQItem` is for: the heap updates it on `Swap`, so each item knows where it lives.

For Dijkstra, the alternative is “lazy decrease-key”: push a new entry with the new priority, skip stale entries on pop. Less bookkeeping, often faster in practice because real heaps usually don’t shrink.

8. Build heap in O(n)

If you have an unsorted slice, `heap.Init` heap-ifies it in O(n), not O(n log n). The classic algorithm: starting from the last non-leaf node, sift down each one.

```
h := &IntHeap{5, 3, 8, 1, 9, 2}
heap.Init(h)
// h is now a valid min-heap
```

Always use `Init` when seeding from existing data. Pushing N elements is O(n log n); `Init` is O(n).

9. Concurrent priority queue

`container/heap` is not safe for concurrent use. Two goroutines pushing concurrently will corrupt the heap.

Simple wrapper:

```go
type SafeHeap struct {
    mu sync.Mutex
    h *IntHeap
}

func (s *SafeHeap) Push(v int) {
    s.mu.Lock()
    defer s.mu.Unlock()
    heap.Push(s.h, v)
}

func (s *SafeHeap) Pop() (int, bool) {
    s.mu.Lock()
    defer s.mu.Unlock()
    if s.h.Len() == 0 { return 0, false }
    return heap.Pop(s.h).(int), true
}
```

For high contention, consider:

- Sharded heaps: route by key hash to N heaps; merge for global queries.
- Concurrent skip-list-based PQ: lock-free, but more complex.
- Channel-based scheduling: if priorities are coarse (high/normal/low), a `select` on multiple channels can suffice.

For most application priority queues, a mutex-wrapped heap is fine.

10. Indexed priority queue

When you need to update an arbitrary item's priority (not just insert/pop), keep a map from key to heap index. The `PQItem` pattern above has this built in via `Index`. Add a map externally:

```go
type IndexedPQ struct {
    items map[string]*PQItem[string]
    pq    *PQ[string]
}

func (q *IndexedPQ) Set(key string, priority int) {
    if item, ok := q.items[key]; ok {
        q.pq.Update(item, key, priority)
        return
    }
    item := &PQItem[string]{Value: key, Priority: priority}
    q.items[key] = item
    heap.Push(q.pq, item)
}

func (q *IndexedPQ) PopMin() (string, bool) {
    if q.pq.Len() == 0 { return "", false }
    item := heap.Pop(q.pq).(*PQItem[string])
    delete(q.items, item.Value)
    return item.Value, true
}
```

This is the structure for Dijkstra with proper decrease-key, OS schedulers with priority changes, simulation event queues.

11. Heap vs sorted structures

Need	Pick
Repeatedly extract min/max	Heap
Random insert + sorted iteration	Sorted slice (if small) or balanced tree
Insert and remove specific values often	Map (no order) or balanced tree
Stream top-K	Min-heap of size K
K-way merge	Heap of K
Bounded “always-keep-best-N” cache	Min-heap with replace-root
Ordered by inserted-at	Slice (it’s already sorted)
Frequent decrease-key	Indexed heap or Fibonacci heap

12. Performance notes

`container/heap` adds modest overhead due to:

- Method calls through an interface (no inlining).
- Boxing into `any` for `Push` / `Pop`.

For tight inner loops, a hand-rolled generic heap is 2-3x faster than the stdlib interface version.

Example specialized min-heap:

```
type IntMinHeap struct { data []int }

func (h *IntMinHeap) Push(v int) {
    h.data = append(h.data, v)
    h.up(len(h.data) - 1)
}

func (h *IntMinHeap) Pop() int {
    v := h.data[0]
    n := len(h.data) - 1
    h.data[0] = h.data[n]
    h.data = h.data[:n]
    if n > 0 { h.down(0) }
    return v
}

func (h *IntMinHeap) up(i int) {
    for i > 0 {
        p := (i - 1) / 2
        if h.data[p] <= h.data[i] { break }
        h.data[p], h.data[i] = h.data[i], h.data[p]
        i = p
    }
}

func (h *IntMinHeap) down(i int) {
    n := len(h.data)
    for {
        l, r := 2*i+1, 2*i+2
        m := i
        if l < n && h.data[l] < h.data[m] { m = l }
        if r < n && h.data[r] < h.data[m] { m = r }
        if m == i { break }
        h.data[i], h.data[m] = h.data[m], h.data[i]
        i = m
    }
}
```

For most code, the stdlib is fast enough. Profile first.

13. Best practices

1. Use `container/heap` for prototypes and non-hot-path code. It works, it's tested, it's small.
2. Hand-roll specialized heaps for hot loops. No interface dispatch, no `any` boxing.
3. Use `heap.Init` to bulk-load. O(n) instead of O(n log n).
4. Pair with an index map when you need decrease-key by key.

5. Lazy delete is often easier than proper remove. Push new entry, skip stale on pop.
6. Heap is not for “sorted output.” It gives you smallest-first; not arbitrary sorted iteration.
7. For concurrent use, wrap with a mutex. Shard or move to channels if contention is high.
8. Stabilize `Less` with a tiebreaker for deterministic ordering in tests.
9. Use a min-heap for top-K largest. Counterintuitive but correct: keep the K largest, evict the smallest.
10. Don’t confuse `heap.Pop` with the interface’s `Pop`. The interface’s `Pop` is internal mechanics.

14. Common gotchas

Gotcha	Symptom	Fix
Calling `(*h).Push` directly	Heap invariant broken	Use `heap.Push(h, x)`
Modifying item priority without Fix	Stale order	`heap.Fix(h, item.Index)`
Forgetting to update Index in Swap	Fix / Remove operate on wrong element	Update both in Swap
Min-heap used for top-K largest	Backwards results	Min-heap is right; root is the threshold
Pop on empty heap	Panic	Check `h.Len() > 0` first
Boxing in stdlib heap (`any`)	Allocation per op	Hand-rolled generic heap
Concurrent access without lock	Race / corruption	Wrap with mutex
heap.Init skipped after bulk load	Invariant violated; weird order	Always Init
Comparing via `==` instead of Less semantics	Stability issues	Add tiebreaker in Less
Tie-broken by pointer address (nondeterministic)	Flaky tests	Use a field-based tiebreaker
Pushing pointer twice without removing	Double-counted, broken Index	Use a map to dedupe before push
Heap as a “priority list” with random access	Not designed for that	Use BST or indexed heap

15. Quick reference

What	Code
Build heap	`heap.Init(h)` (O(n))
Push	`heap.Push(h, x)`
Pop min/max	`heap.Pop(h)`
Peek	`(*h)[0]` (without removing)

What	Code
Repair after edit	`heap.Fix(h, i)`
Remove arbitrary	`heap.Remove(h, i)`
Interface	`sort.Interface` + `Push(any)` + `Pop() any`
Min-heap	`Less(i,j) = h[i] < h[j]`
Max-heap	`Less(i,j) = h[i] > h[j]`
Tie-break	Add secondary condition in `Less`
Index field	Updated in `Swap`; used by Fix / Remove
Lazy delete	Push new entry, skip stale on pop
Concurrent	Wrap with `sync.Mutex`
Generic	`PQItem[T]` with Priority field
Top-K	Min-heap of size K
Dijkstra	Push (dist, node); skip stale on pop
K-way merge	Heap of (value, stream-index)
Median	Two heaps (max for lower, min for upper)
Build complexity	O(n) for Init
Push/Pop complexity	O(log n)
Peek complexity	O(1)